Phase 4 user flow: evidence and digital twin

Use **Evidence & digital twin** in ArchIntel. This phase adds immutable architecture versions, canonical evidence/facts, guided runtime investigation and a persistent graph. Existing static/runtime analysis continues to work independently.

Capture a version

1. Sign in and open **Evidence & digital twin**. Select the application and a completed static baseline.
2. Enter a descriptive version name. Optionally choose a parent version to record lineage.
3. Select completed runtime sessions from the same static baseline. Leave the selection empty for a static-only version. A running session cannot be frozen into a version; stop it first in Dynamic analysis. At most ten sessions can be selected.
4. Optionally enable **Review domain names and membership**. Give a review reason, edit domain names and assign classes using the selectors. Existing domain IDs stay stable, every class must be assigned once, and no domain can be empty. The UI shows the first 300 classes; the validated API supports the full bounded population. Changing domains creates a new version; it never rewrites the original analysis or recalculates modernization scores.
5. Click **Preview architecture version**. Review fact/evidence/node counts and the fingerprint. Changing any form value invalidates the preview.
6. Click **Save architecture version**. An identical baseline/session selection/domain review/parent returns the existing version, even if you enter a different display name. The fingerprint identifies the evidence/configuration, not the name.
7. Select the saved version using **Architecture version**. The scope panel shows baseline, selected runtime sessions, derivation version and the last graph-projection state.

An account needs ANALYSIS_WRITE to capture/review versions or update discovery, and ANALYSIS_READ to inspect them. Versions containing runtime evidence also require RUNTIME_READ. Access remains restricted to the account's workspace. Runtime-only permission does not grant access to the combined architecture workspace.

Inspect facts and provenance

Open **Facts**. Each directed dependency is classified as:

- **STATIC_ONLY:** present in bytecode evidence, not observed in the selected runtime graph. This never means unused.
- **DYNAMIC_ONLY:** observed at runtime without the corresponding internal static dependency.
- **STATIC_AND_DYNAMIC:** supported by both sources.

Click **Inspect evidence** on a fact. Expand each supporting observation to see collector, timestamp, source/target, captured fields and available span links. Aggregated evidence remains available when raw traces expire. Such span references are explicitly marked EXPIRED_OR_UNAVAILABLE rather than silently pointing to missing data.

The evidence record timestamp is the version's derivation time. Runtime source observations retain their captured timestamps inside the observation. Class/method identity remains conservative; unsupported or ambiguous mappings are not forced. The displayed evidence confidence describes the adapter's supporting observation, not a calibrated modernization probability.

Plan guided runtime discovery

Open **Discovery** to review prioritized suggestions for unobserved entry points, scheduled jobs, message consumers, dependencies, domains and high-risk findings. UNKNOWN means reliable signature-level observation is unavailable, for example after raw retention expiry. Suggestions do not claim to know a complete business workflow.

Enter a **Discovery update note**, then change an item's status to OPEN, PLANNED, EXERCISED or DISMISSED. Expand Update history to review previous changes. Concurrent changes require a refresh before saving.

EXERCISED records your workflow progress; it does not increase coverage. To validate the outcome, create a new runtime session, exercise the intended routes, stop it and capture a successor architecture version. Choose the earlier version as parent. **Compare observations with parent** shows percentage-point differences only when baseline and domain review are compatible. It does not claim that modernization improved or that a particular action caused the change.

Inspect the persistent graph and twin

The graph is a separate authenticated Neo4j service on local port 17687; it is not the temporary scanner graph. Projection normally runs in the background. Use **Rebuild graph projection** to replay the saved canonical data immediately.

Open **Graph**. A CURRENT result confirms that the stored version digest and projection counts match. The UI displays a bounded sample, and the neighborhood selector follows one hop around a selected node. The graph includes version/application/session references, domains, packages, classes, methods/entry points, external type references, observed resources/flows, evidence, facts and existing findings/cycles.

Open **Twin** for version scope, reviewed domains, observation coverage and explicit unknowns. Database calls do not prove ownership. Tests, behavioral contracts and modernization execution state are not invented. Static readiness scores and existing finding records remain baseline outputs; Phase 5 assessments are not generated here.

Use **Export canonical evidence** to download/read the complete bounded version document with evidence and facts. Discovery progress is mutable operational history and is retrieved separately through the discovery API; it does not modify that frozen export.

Troubleshooting

State or symptom	Meaning and action
PENDING	Version committed; background projection has not completed. Wait or rebuild.
FAILED	Projection attempt failed. Canonical facts/evidence remain available. Restore the local graph connection and rebuild.
APPLIED but graph cannot open	APPLIED describes the last successful write. The graph query checks its present availability/currentness.
HTTP 409 stale/incomplete graph	Rebuild this version. Its canonical records are retained.
HTTP 503 graph unavailable	Run the local graph startup command, inspect its logs and retry. Static analysis/evidence do not depend on graph availability.
Capture rejected for raw retention	Expired sessions retain individual reports, but cannot be recombined into a new multi-session report. Capture one archived session or select retained raw sessions.
Method coverage unavailable	Do not infer signature coverage from aggregate counts. Use a new method inventory and a fresh retained runtime capture.
Version comparison rejected	Use the same static baseline and compatible domain-review scope.
Discovery update conflict	Refresh; another update changed its revision. Reapply the intended status with a new note.

Local lifecycle

`Start-ArchIntel.cmd` starts the configured persistent graph along with the app. `Stop-ArchIntel.cmd` stops the recorded ArchIntel graph JVM after verifying its identity. Data persists under `.data/knowledge-graph`; do not delete or share its password key. The key is unrelated to the administrator bootstrap key.

For a prepared installation, from the repository root:

```
.\.runtime\python\tools\python.exe scripts/knowledge_graph.py start
.\.runtime\python\tools\python.exe scripts/knowledge_graph.py check
```

See Phase 4 verification and developer handoff. Phase 5 has not started.